

ScriptableObjects Tutorial

Preamble.....	2
ScriptableObjects.....	2
Mothership Repo.....	2
How To.....	3
Part 1: The Code.....	3
Part 2: The Unity Assets.....	8
Part 3: A Small Bit of Refactoring.....	12
Recap.....	16

Preamble

In Mothership, the player controls a mining drone that undocks from the ship and the player can fly around to shoot their mining laser at asteroids to mine them. The player earns credits based on the asteroid's value. The player has a limited amount of fuel to use before they must dock again.

Credits earned from mining asteroids can be spent on upgrades by clicking a button in a menu while the player is docked back at the ship.

This is the basic game loop; undock, earn money before time runs out, dock, buy upgrades, undock.. .

The player's drone has various attributes or 'stats'; health, speed, boost speed that upgrades apply to.

These base player stats and the upgrades are scriptable objects and managed by a stat manager script.

The stats for speed and boost speed are already in the game and upgradeable via the menu.

I'm going to walk through how I'll add an upgrade to the player's health stat. It will require two new scriptable objects and a TextMeshPro button.

I will explain the scripts that manage the changes to the players stats as we go. There will be a small amount of refactoring.

We also need to understand [ScriptableObjects](#).

ScriptableObjects

The player has stats, these would typically be contained in the player class, with variables for health, speed etc. These values are either hardcoded in the class or assigned at runtime. This is fine, but tedious to change and adjust, and magic numbers are not a clean coding practice. Also, with adjustments to values during the game, it may become complicated, and whatever script is managing it will become monolithic, at worst.

So where do we store the variables?

We can use ScriptableObjects. They act like data storage containers that we can reference and pull values from. They are easily adjusted within the unity editor, so you don't have to go diving into the code base to change values.

Mothership Repo

This is the branch containing the scripts from this tutorial, you can see the scripts without (some of) the comments.

<https://github.com/DesmondShearer/mothership/commits/scriptableobjects-tutorial/>

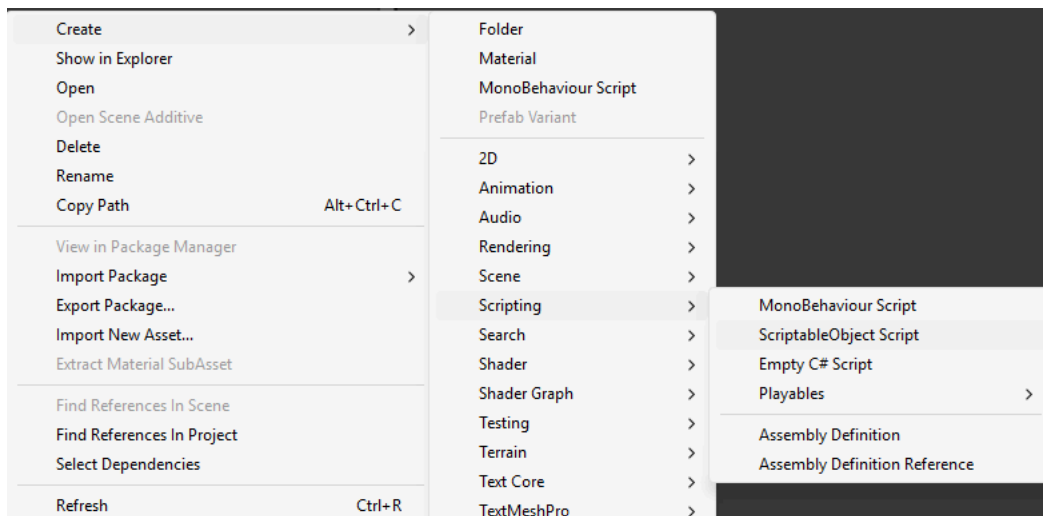
How To

Part 1: The Code

Let's create a ScriptableObject that will handle a player stat and upgrade.

In the Scripts folder:

- Right Click > Create > Scripting > ScriptableObject



- Rename the ScriptableObject - **PlayerStat**
- Add the following code:

```
using UnityEngine;

// This allows for player stats to be created as assets within Unity Editor.
[CreateAssetMenu(fileName = "PlayerStat", menuName = "Scriptable
Objects/PlayerStat")]

public class PlayerStat : ScriptableObject
{
    public string statName;
    public float baseValue;
}
```

Doing the same again, but this time creating a ScriptableObject for upgrading a stat, note we reference the base player stat ScriptableObject.

In the Scripts folder:

- Right Click > Create > Scripting > ScriptableObject

- Name the ScriptableObject - **PlayerStatUpgrade**
- Add the following code:

```
using UnityEngine;

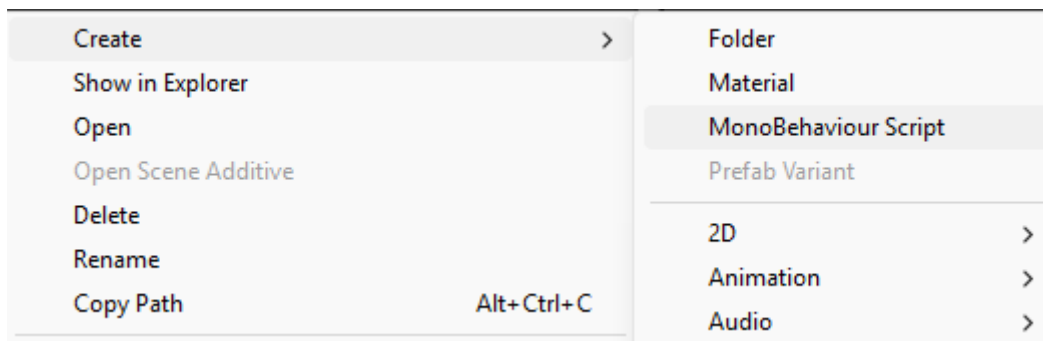
[CreateAssetMenu(fileName = "PlayerStatUpgrade", menuName = "Scriptable
Objects/PlayerStatUpgrade")]

public class PlayerStatUpgrade : ScriptableObject
{
    public PlayerStat targetStat;
    public float increaseAmount;
    public int upgradePrice;
}
```

Now we need to write a script to manage the player stats.

In the Scripts folder:

- Right Click > Create > MonoBehaviour Script



- Name the MonoBehaviour Script - **PlayerStatManager**
- Add the following code:

```
using System.Collections.Generic;
using UnityEngine;

public class PlayerStatManager : MonoBehaviour
{
    // This will allow us to view the players current stats
    // during play mode and aids with debugging.
    [System.Serializable]
    public class StatsAtRuntime
    {
        public PlayerStat playerStat;
    }
}
```

```

        public float currentValue;
    }

    // This List will hold our player stats.
    public List<StatsAtRuntime> playerStats = new List<StatsAtRuntime>();
    // The credit manager allows us to change the bank balance.
    public CreditManager creditManager;

    void Start()
    {
        // On start assign stats to player.
        InitializeStats();
    }

    void InitializeStats()
    {
        // for each stat in the List, assign the stat from the ScriptableObject
        foreach (var s in playerStats)
        {
            s.currentValue = s.playerStat.baseValue;
        }
    }

    public float GetStatValue(PlayerStat stat)
    {
        // In the List, look for stat we've asked for, if not
        // found return 0
        var found = playerStats.Find(s => s.playerStat == stat);
        return found != null ? found.currentValue : 0;
    }

    public void ApplyUpgrade(PlayerStatUpgrade playerStatUpgrade)
    {
        // check if player has enough credits to upgrade, if not,
        // don't upgrade
        if (creditManager.totalCredits < playerStatUpgrade.upgradePrice)
        {
            return;
        }
        // find the stat to upgrade
        var stat = playerStats.Find(s => s.playerStat ==
playerStatUpgrade.targetStat);
        // if found, upgrade the stat and adjust credit balance
        if (stat != null)
        {
            stat.currentValue += playerStatUpgrade.increaseAmount;
            creditManager.UpdateCredits(-playerStatUpgrade.upgradePrice);
        }
    }

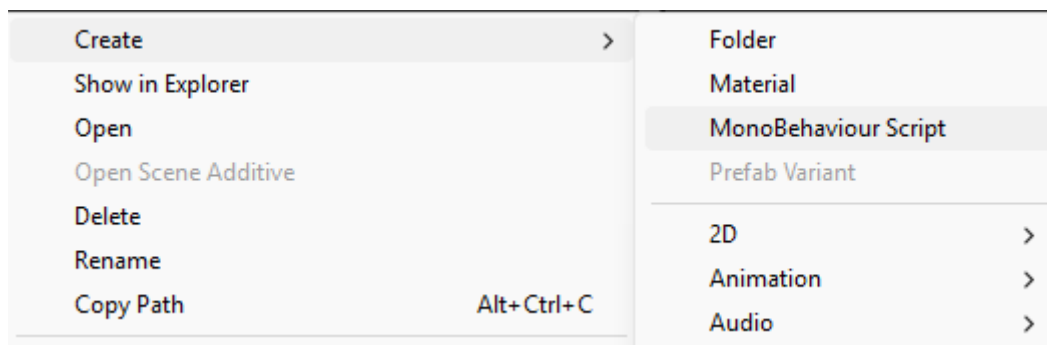
```

```
}  
}  
}
```

Now we need to create a script to manage the button interaction. This will trigger the methods from the PlayerStatManager.

In the Scripts folder:

- Right Click > Create > MonoBehaviour Script



- Name the MonoBehaviour Script - **PlayerStatUI**
- Add the following code:

```
using TMPro;  
using UnityEngine;  
using UnityEngine.UI;  
  
public class StatUI : MonoBehaviour  
{  
  
    // This allows us to assign the specific assets in the Unity Editor  
    // a button  
    public PlayerStatManager playerStatManager;  
  
    public TextMeshProUGUI statText;  
    public TextMeshProUGUI priceText;  
    public Button upgradeButton;  
  
    public PlayerStatUpgrade playerStatUpgrade;  
  
    private void Start()  
    {
```

```

        // add a listener to the button, when clicked, try an upgrade
        upgradeButton.onClick.AddListener(OnUpgradeClicked);
        // update the ui to show values
        UpdateUI();
    }

    void UpdateUI()
    {
        // find the values to display
        playerStatManager.GetStatValue(playerStatUpgrade.targetStat);

        statText.text = $"{playerStatUpgrade.targetStat.statName}
+{playerStatUpgrade.increaseAmount}";
        priceText.text = $"${playerStatUpgrade.upgradePrice}";
    }

    void OnUpgradeClicked()
    {
        // update chosen stat, update the ui
        playerStatManager.ApplyUpgrade(playerStatUpgrade);
        UpdateUI();
    }
}

```

Part 2: The Unity Assets

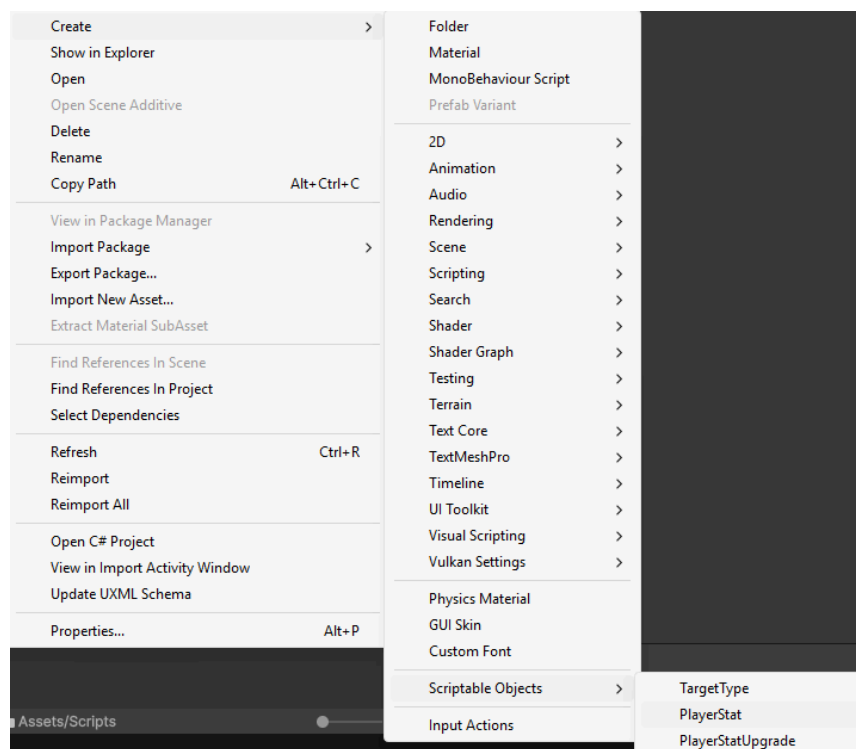
Now that we have our scripts ready to go, let's create the assets that are used by them.

We will need:

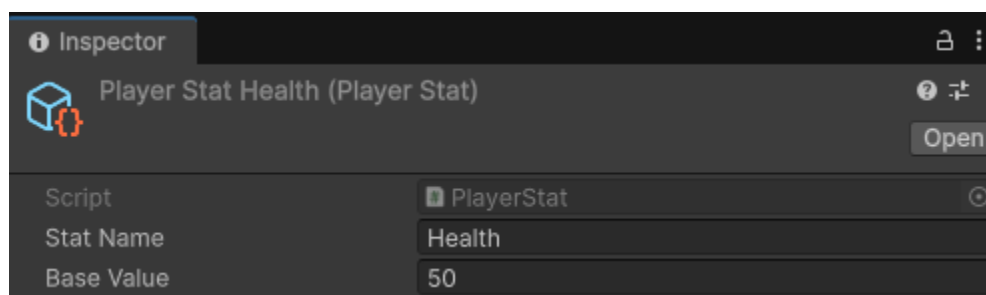
- A PlayerStat ScriptableObject for health - **PlayerStatHealth**
- A PlayerStatUpgrade ScriptableObject for health - **PlayerStatUpgradeHealth**

In the Scripts folder:

- Right Click > Create > Scriptable Objects > PlayerStat



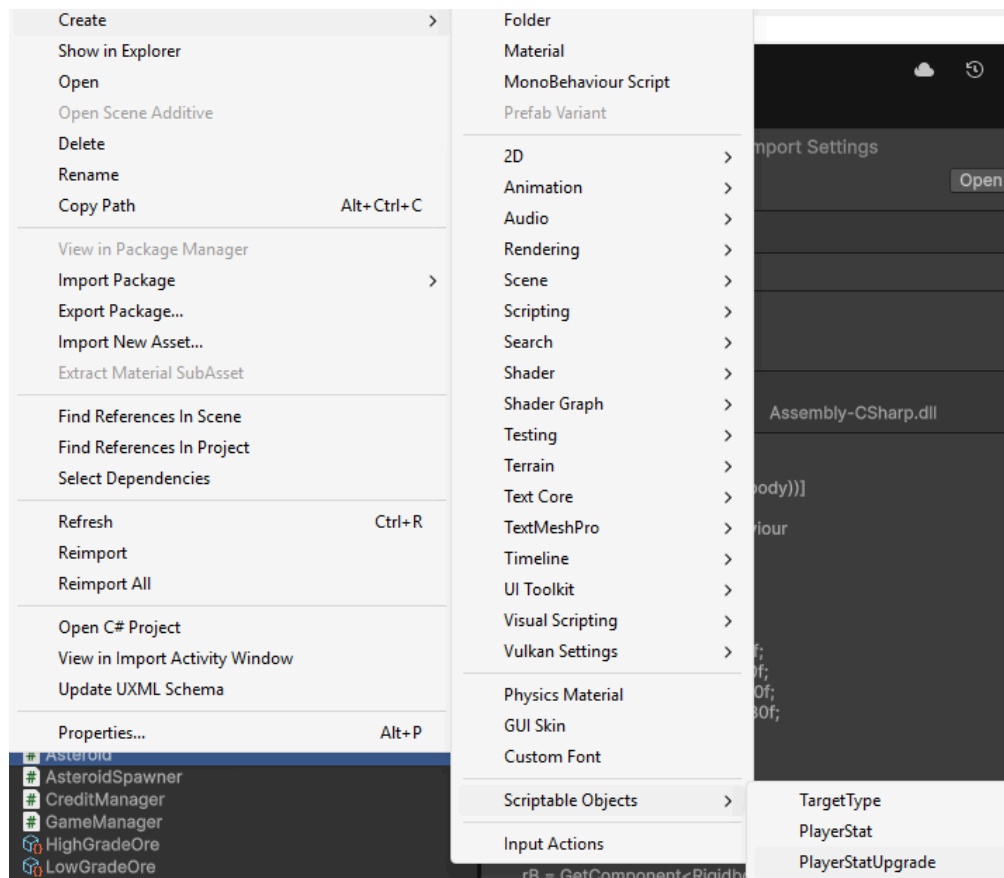
- Name it **PlayerStatHealth**
- Assign values in the inspector



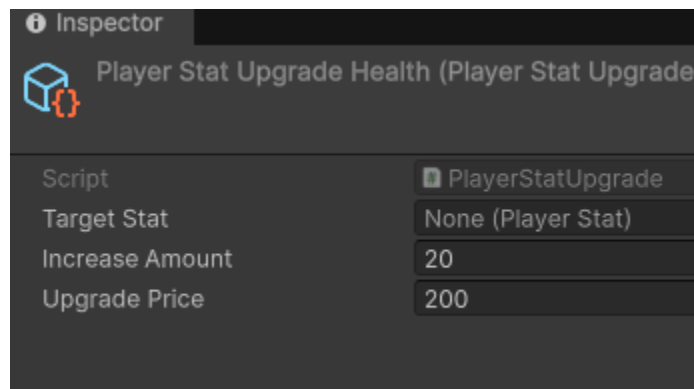
Repeat this for the PlayerStatUpgrade upgrade.

In the Scripts folder:

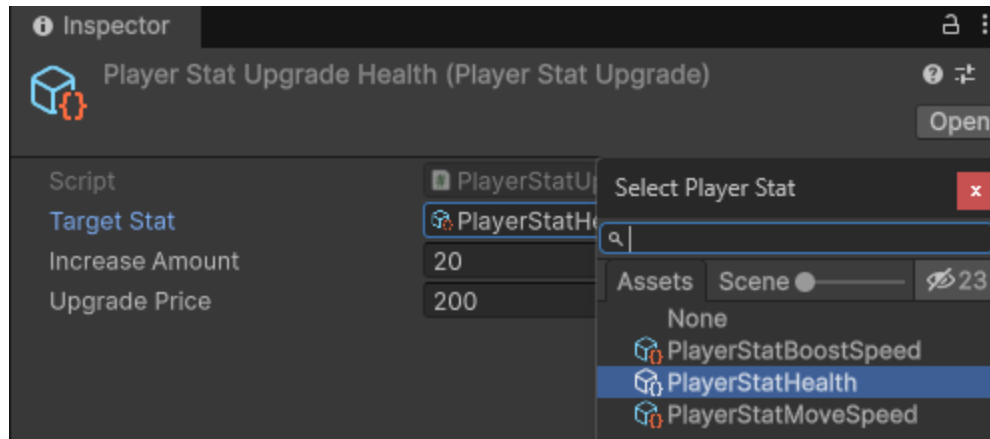
- Right Click > Create > Scriptable Objects > PlayerStatUpgrade



- Name it **PlayerStatUpgradeHealth**
- Assign values in the inspector

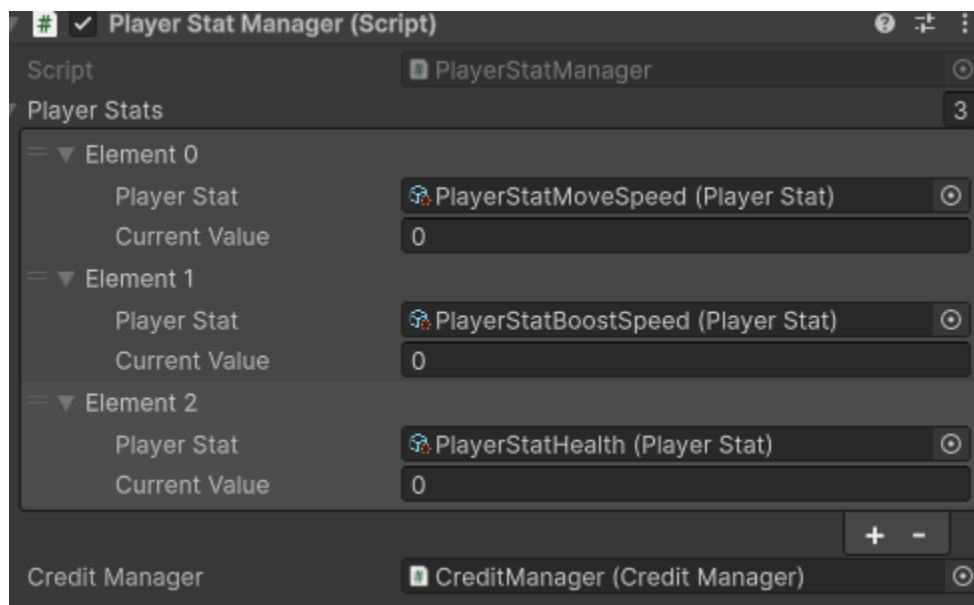


- Assign PlayerStatHealth ScriptableObject to the Target Stat



Attach the PlayerStatManager to the Player object by using the Add Component button with the player object selected or dragging the PlayerStatManager script onto it.

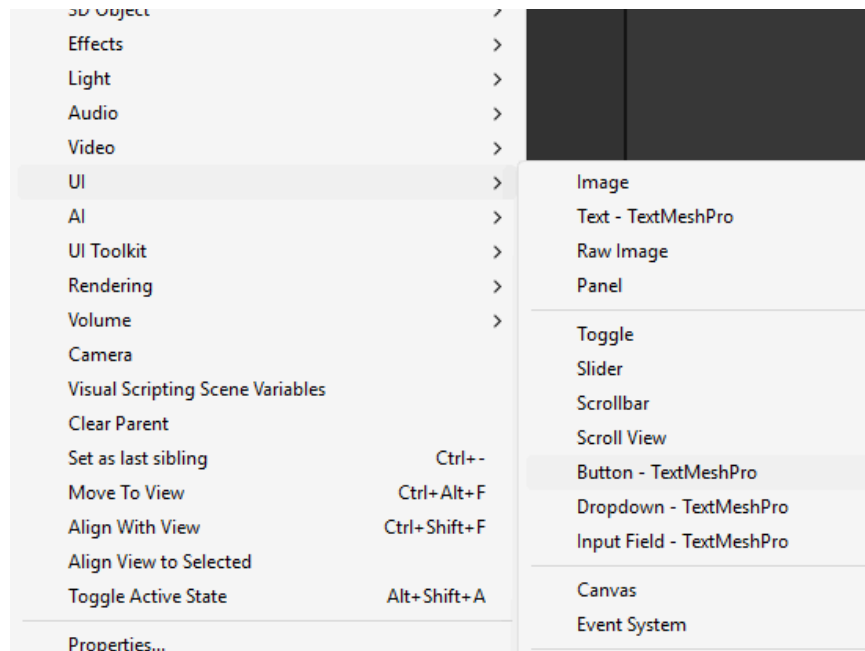
Once attached, populate the List with stats you've created and assign the credit manager.



NOTE - the current values are zero as the game is not running.

In Canvas in the scene:

- Right Click > UI > Button - TextMeshPro
- Name the button - UpgradeHealth

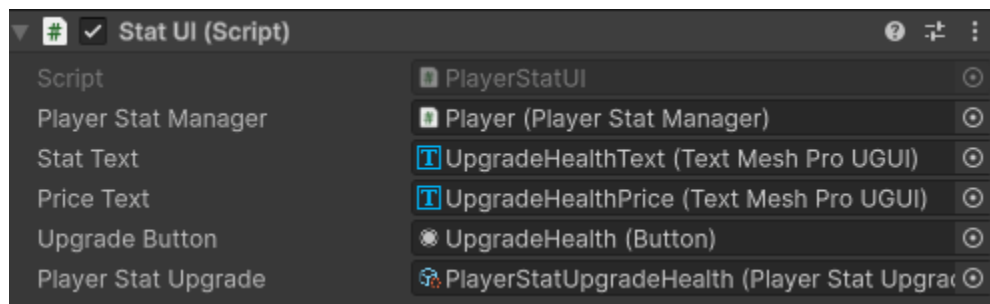


Select the UpgradeHealth button and add another text field:

- Right Click > UI > Text - TextMeshPro
- Rename both text fields to **UpgradeHealthText** and **UpgradeHealthPrice**

Align the buttons for ease of use and then attach the StatUI script to the button using Add Component with the button selected or by dragging the script on to the button.

Link up the assets.



Part 3: A Small Bit of Refactoring

The PlayerStatManager will now return the current health value, so we need to adjust the GameManager code to handle it.

I removed the hardcoded variable for the player's health and it now reads from the PlayerStatManager.

The GameManager needs to be linked up in Unity, so I added references for the PlayerStatManager and the playerStatHealth scriptable object.

```
using System;
using UnityEngine;
using System.Collections.Generic;
using TMPro;
using UnityEngine.UI;
using Random = UnityEngine.Random;

public class GameManager : MonoBehaviour
{
    // Removed variables here for initial and
    // current player health
    private float initialPlayerFuel = 100;
    private float currentPlayerFuel;

    // add references for the PlayerStatManager
    // and player health stat ScriptableObject
    public PlayerStatManager playerStatManager;
    public PlayerStat playerStatHealth;

    public TextMeshProUGUI healthText;
    public TextMeshProUGUI fuelText;

    public PlayerController playerController;

    public CreditManager creditManager;

    public Image menu;
    public TextMeshProUGUI titleText;
    public TextMeshProUGUI instructionsText;

    public Image gameOver;
    public TextMeshProUGUI gameOverText;
    public TextMeshProUGUI gameOverReasonText;
    public TextMeshProUGUI gameOverScoreText;
```

```

public Image upgradesMenu;

public bool gameOverCheck;

public AudioSource dieAudio;
public bool alreadyPlayed = false;

void Start()
{
    gameOverCheck = false;
    gameOver.gameObject.SetActive(false);
    gameOverText.gameObject.SetActive(false);
    gameOverReasonText.gameObject.SetActive(false);
    gameOverScoreText.gameObject.SetActive(false);
}

void Update()
{
    // Display health function added
    DisplayFuel();
    DisplayHealth();

    if (playerController.isDocked)
    {
        menu.gameObject.SetActive(true);
        titleText.gameObject.SetActive(true);
        instructionsText.gameObject.SetActive(true);
        upgradesMenu.gameObject.SetActive(true);
        Cursor.visible = true;

        currentPlayerFuel = initialPlayerFuel;
    }

    if (!playerController.isDocked)
    {
        menu.gameObject.SetActive(false);
        titleText.gameObject.SetActive(false);
        instructionsText.gameObject.SetActive(false);
        upgradesMenu.gameObject.SetActive(false);
        Cursor.visible = false;

        currentPlayerFuel -= Time.deltaTime;
    }
}

```

```

    }

    // Game over check now references the player stat manager
    if (playerStatManager.GetStatValue(playerStatHealth) <= 0)
    {
        GameOver();
        gameOverReasonText.text = "You took too much damage!";
    }

    if (currentPlayerFuel <= 0)
    {
        GameOver();
        gameOverReasonText.text = "The fuel tank is empty!";
    }
}

// Removed this function - it's been moved to PlayerStatManager

// public void RemoveHealth(int damageToTake)
// {
//     currentPlayerHealth -= damageToTake;
//     healthText.text = "Health: " + currentPlayerHealth;
// }

// Get the health from player stat manager and
// display it
public void DisplayHealth()
{
    float health = playerStatManager.GetStatValue(playerStatHealth);
    healthText.text = "Health: " + health.ToString("000");
}

public void DisplayFuel()
{
    fuelText.text = "Fuel: " + currentPlayerFuel.ToString("000");
}

public void GameOver()
{
    gameOverCheck = true;
    gameOver.gameObject.SetActive(true);
    gameOverText.gameObject.SetActive(true);
    gameOverReasonText.gameObject.SetActive(true);
    gameOverScoreText.text = "You earned " + creditManager.totalCredits + "
Credits!";
    gameOverScoreText.gameObject.SetActive(true);

```

```

        if (!alreadyPlayed)
        {
            dieAudio.Play();
            alreadyPlayed = true;
        }
    }
}

```

The TakeDamage function added to the PlayerStatManager script replaces the RemoveHealth function.

```

public void TakeDamage(PlayerStat playerStatHealth, float damageToTake)
{
    var stat = playerStats.Find(s => s.playerStat == playerStatHealth);

    if (stat != null)
    {
        stat.currentValue -= damageToTake;
        // Clamp to 0 (no negative health)
        stat.currentValue = Mathf.Max(stat.currentValue, 0);
    }
}

```

And lastly, the collision detector on the PlayerController script was updated to reference the playerstat manager also.

```

private void OnCollisionEnter(Collision collision)
{
    if (collision.gameObject.CompareTag("Asteroid"))
    {
        damageAudio.Play();
        // This used to reference the gameManager.RemoveHealth()
        playerStatManager.TakeDamage(playerHealth,10);
    }
}

```

Recap

I hope this has made some sense. It took me a considerable amount of time to gain an understanding of how to do this.

We can now add new stats to the player along with upgrades for them.

We can create buttons and menus to enhance gameplay and provide extensions to the game loop.

We refactored parts of the GameManager and moved functions to more appropriate scripts.

Take Homes:

ScriptableObjects act as data stores that don't change during gameplay.

Think of them as a 'source of truth'.

At runtime a manager script creates a copy and it can be modified safely.